Graph Traversal - Breadth First Search (BFS)

Breadth-First Search (BFS) is a graph traversal algorithm that explores a graph's vertices (nodes) level by level. It starts from a selected source node and moves outward to visit all the nodes at the same distance from the source before moving on to nodes at the following distance level.

BFS is particularly useful for finding the shortest path in unweighted graphs and for systematically exploring graphs.

Here is a complete description of the Breadth-First Search algorithm:

1. Use a Queue

- BFS uses a queue to keep track of the nodes to be visited. The queue follows the First-In-First-Out (FIFO) principle, where the first node inserted in the queue will be the first one to be removed (dequeue).

2. Initialization:

- Start by selecting a source node to begin the traversal.
- Create an empty queue to hold the nodes to be visited.
- Mark the source node as visited and enqueue it into the queue.

3. Traversal:

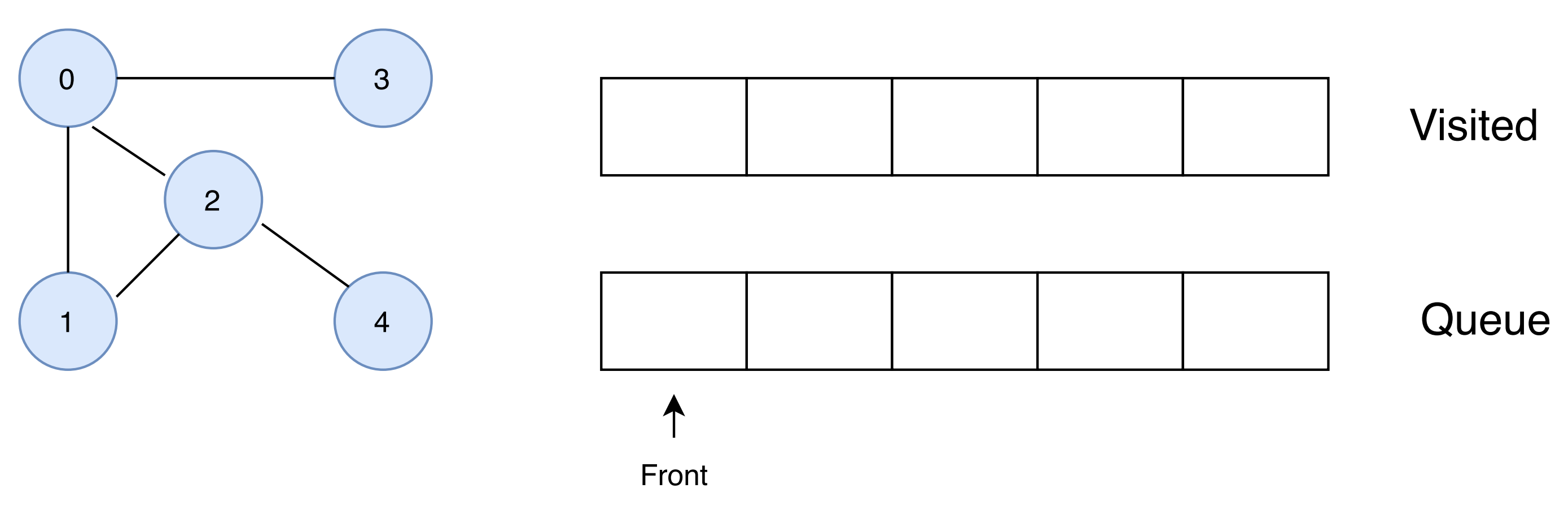
- While the queue is not empty, continue the following steps:
  - Dequeue a node from the front of the queue (let's call it the "current node").
  - Process the current node (print it, perform some operation, etc.).
  - Enqueue all the unvisited neighbors of the current node into the queue.
  - Mark each enqueued neighbor as visited.

4. Termination:

- The BFS algorithm continues until the queue becomes empty, meaning all reachable nodes from the source node have been visited.

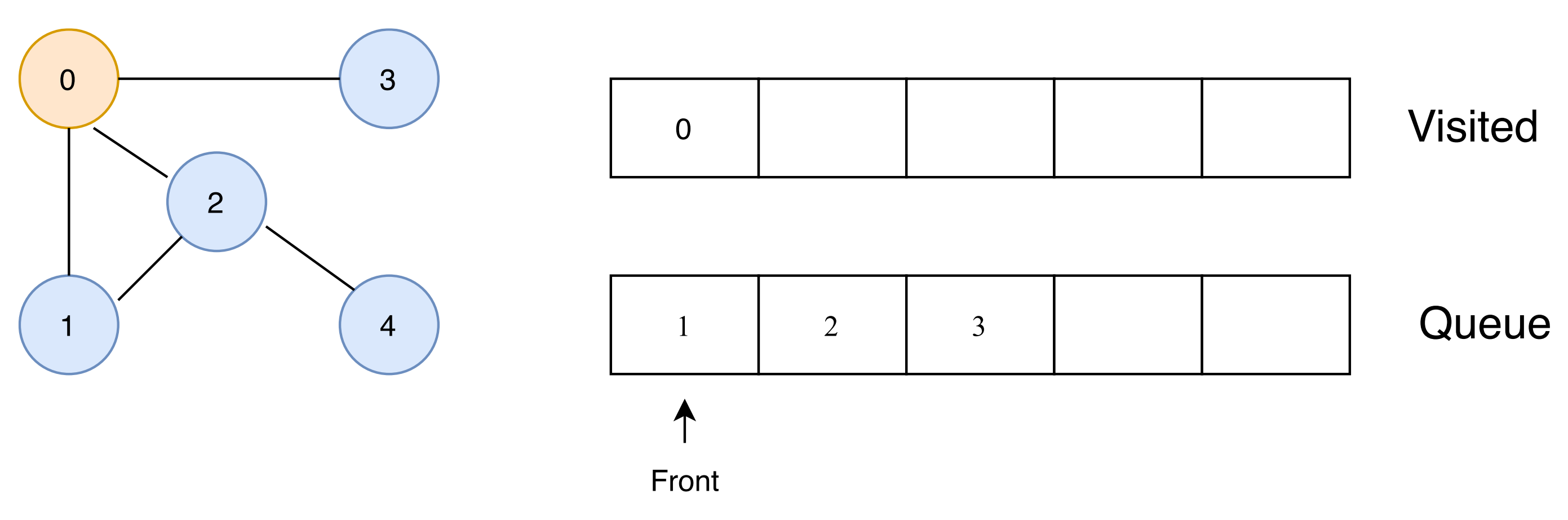
Step-by-Step Illustration of BFS

Let's see how the Breadth First Search algorithm works with an example. We use an undirected graph with 5 vertices.



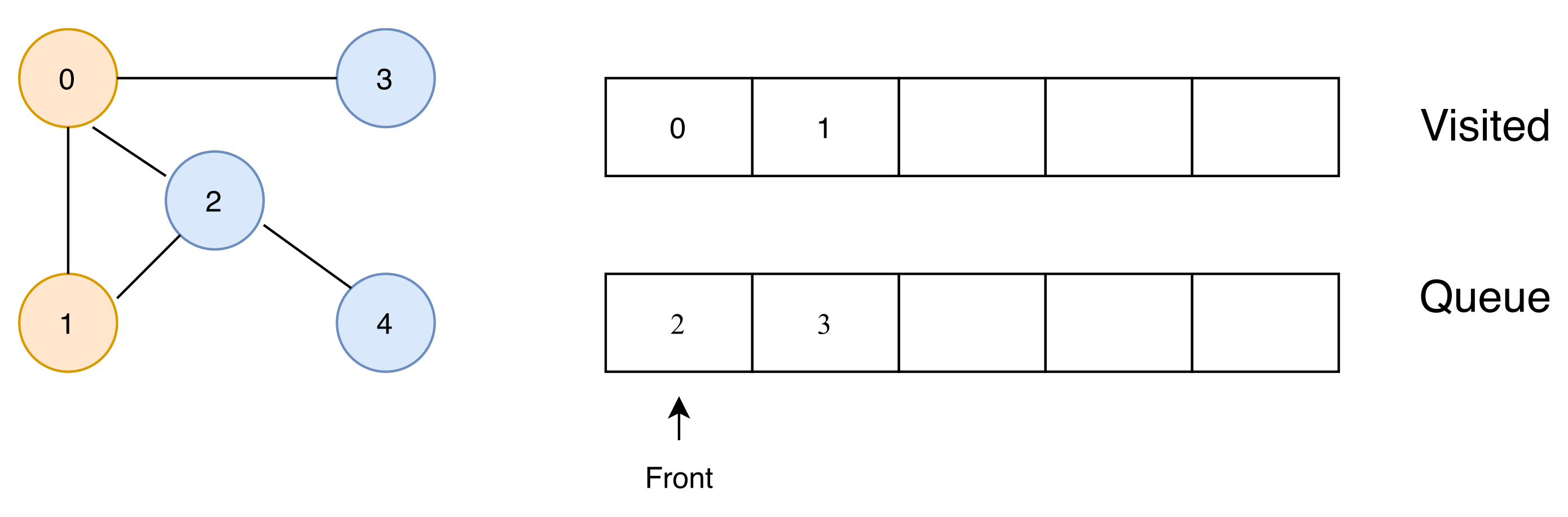
Breadth First Search (BFS)

We start from vertex 0, the BFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the queue.



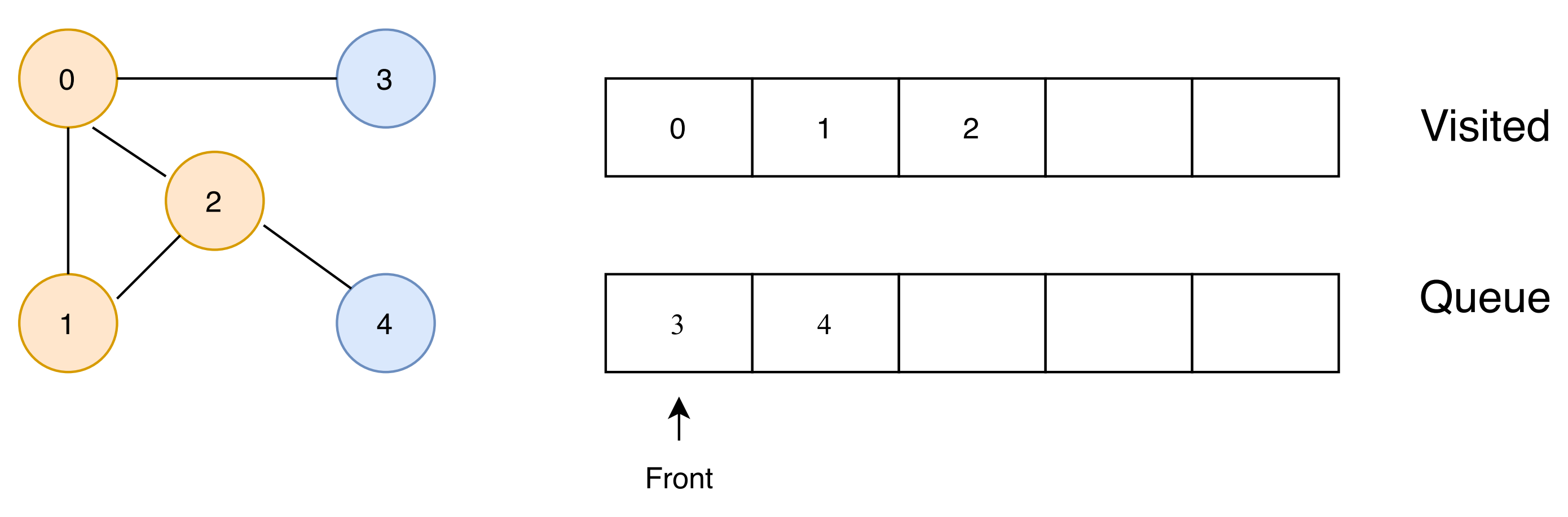
Breadth First Search (BFS)

Next, we visit the element at the front of queue i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.

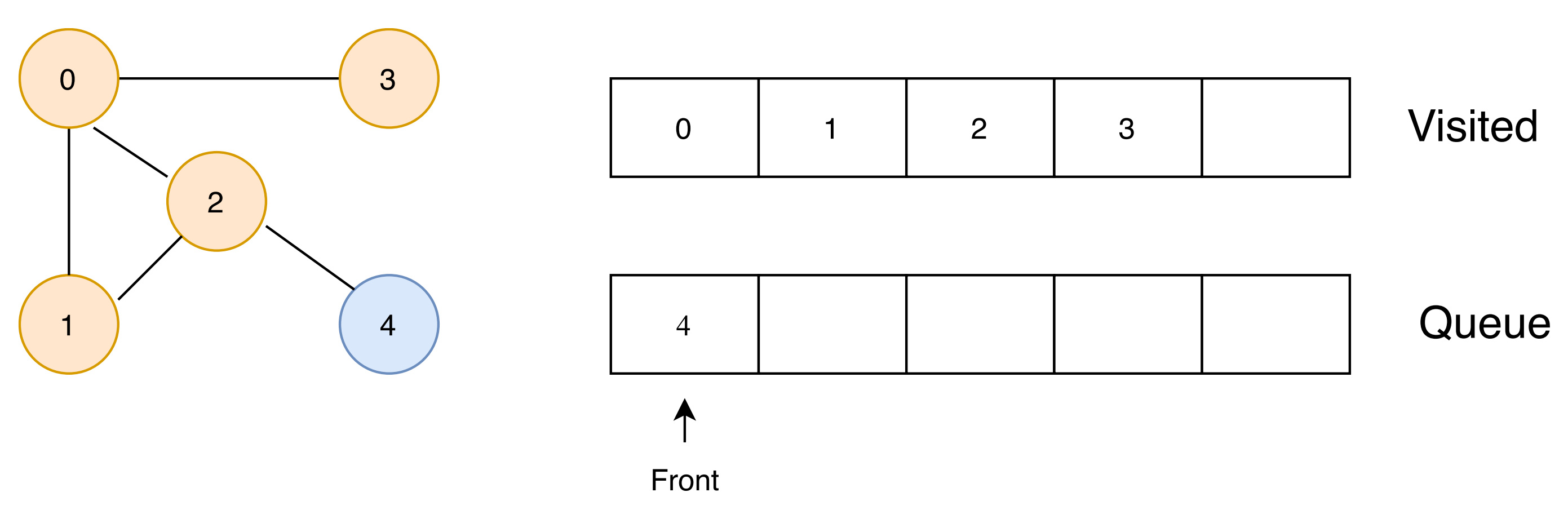


Breadth First Search (BFS)

Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the back of the queue and visit 3, which is at the front of the queue.

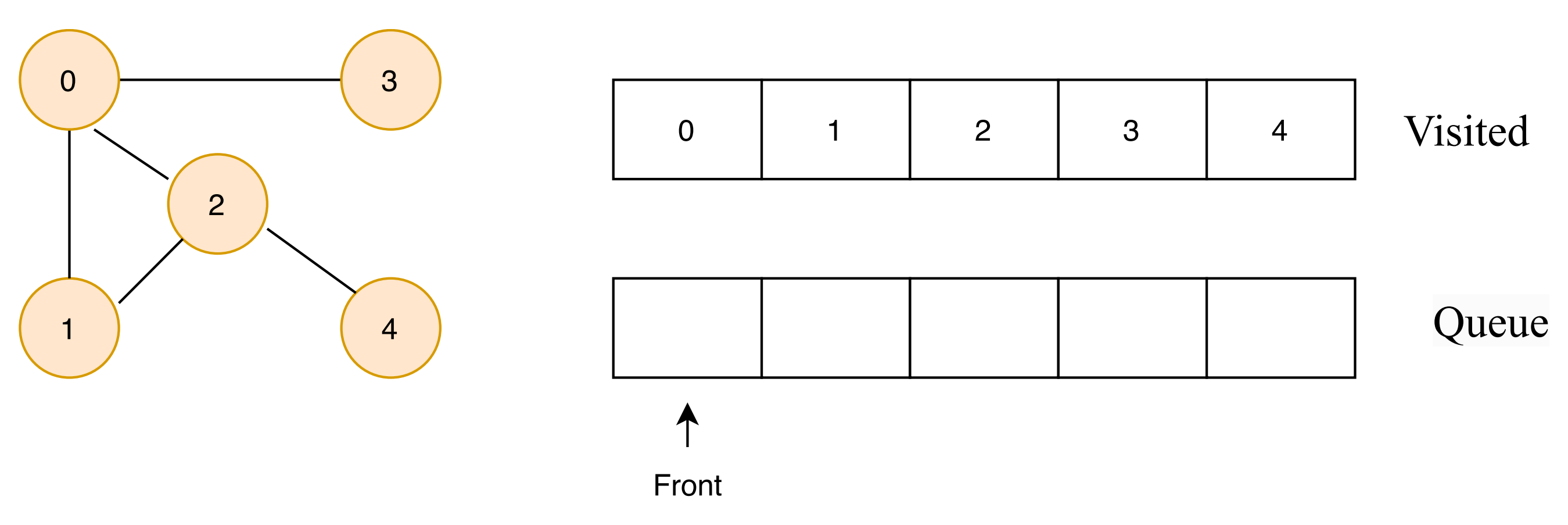


Breadth First Search (BFS)



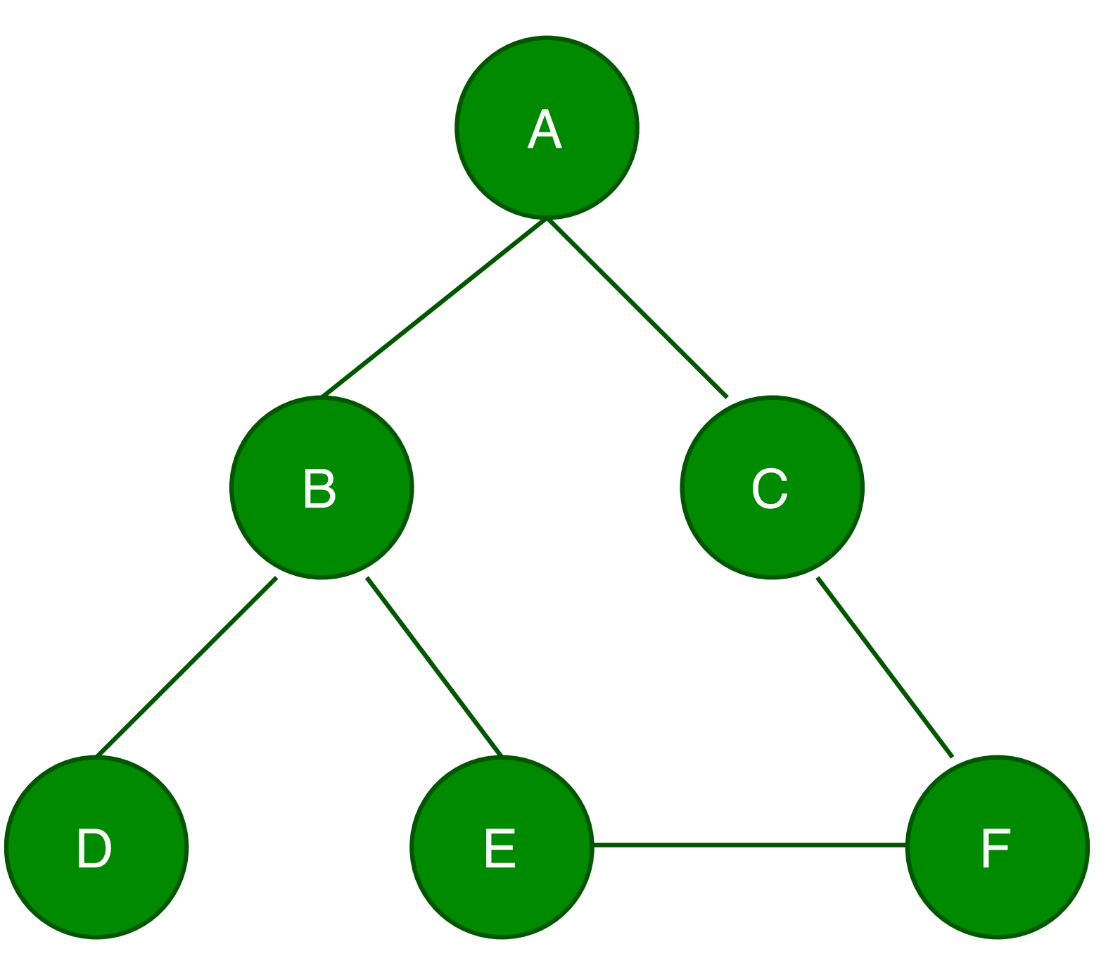
Breadth First Search (BFS)

Only 4 remains in the queue since the only adjacent node of 3 i.e. 0 is already visited. We visit it.



Breadth First Search (BFS)

Now, consider this example:



Starting from the source node A, the BFS traversal would be as follows:

1. Start with A (the source node) and enqueue it. Queue: [A]
2. Dequeue A and process it. Enqueue its neighbors B and C. Queue: [B, C]
3. Dequeue B and process it. Enqueue its neighbors D and E. Queue: [C, D, E]
4. Dequeue C and process it. Enqueue its neighbor F. Queue: [D, E, F]
5. Dequeue D and process it. No unvisited neighbors to enqueue. Queue: [E, F]
6. Dequeue E and process it. Enqueue its neighbor F. Queue: [F]
7. Dequeue F and process it. No unvisited neighbors to enqueue. Queue: []

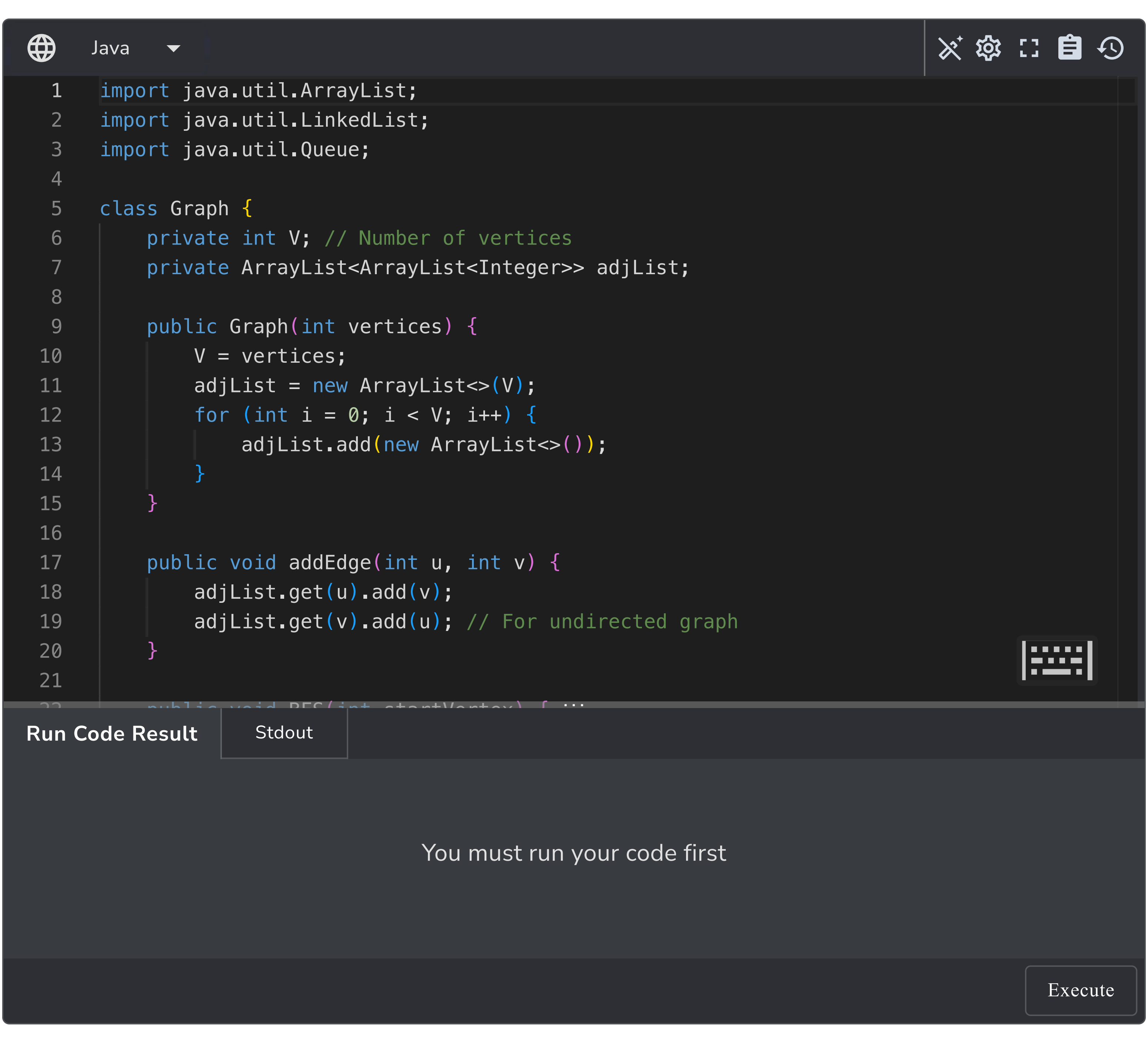
The BFS traversal order is: A -> B -> C -> D -> E -> F.

BFS guarantees that it visits nodes in the order of their distance from the source node.

It is an efficient algorithm to find the shortest path in unweighted graphs. Additionally, BFS can be used to find connected components, detect cycles, and solve various graph-related problems. However, it may consume more memory compared to DFS, especially in graphs with a large branching factor or infinite graphs.

Implementation of Breadth First Search

Below is an implementation of Breadth-First Search (DFS) in C++, Python, Java, and JavaScript. BFS is an algorithm used for traversing or searching a graph or tree in a level-by-level manner.



```java
import java.util.ArrayList;
import java.util.LinkedList;
import java.util.Queue;

class Graph {
    private int V; // Number of vertices
    private ArrayList<ArrayList<Integer>> adjList;

    public Graph(int vertices) {
        V = vertices;
        adjList = new ArrayList<>(V);
        for (int i = 0; i < V; i++) {
            adjList.add(new ArrayList<>());
        }
    }

    public void addEdge(int u, int v) {
        adjList.get(u).add(v);
        adjList.get(v).add(u); // For undirected graph
    }

```

You must run your code first

Complexity analysis

The time and space complexity of Breadth-First Search (BFS) depends on the size of the graph and the way it is represented. Let's analyze the complexities:

1. Time Complexity:

- Visiting a vertex takes $O(1)$ time as we dequeue it from the queue in constant time.
- Exploring the neighbors of a vertex takes $O(1)$ time per neighbor, as we have to traverse its adjacency list once.
- In the worst case, we visit all the vertices at least once, which takes $O(V)$ time. Additionally, for each vertex, we explore all its neighbors once, which takes $O(E)$ time in total (sum of the sizes of all adjacency lists).
- Hence, the overall time complexity of BFS is: $O(V+E)$

2. Space Complexity:

- The space required to store the graph using an adjacency list representation is $O(V+E)$, as we need to store each vertex and its corresponding edges.
- The space required for the queue in BFS is $O(V)$ in the worst case, as all the vertices can be in the queue at once.
- Since the space occupied by the queue is dominant in the overall space complexity, the space complexity of BFS is: $O(V)$

BFS is generally efficient for searching and traversal when the graph is not too dense. For sparse graphs, where E is much smaller than V^2, the time complexity becomes almost linear, making BFS a reasonable choice for many practical applications.

BFS guarantees it visits nodes according to their distance from the source node. It is an efficient algorithm to find the shortest path in unweighted graphs. Additionally, BFS can find connected components, detect cycles, and solve graph-related problems. However, it may consume more memory than DFS, especially in graphs with a significant or infinite branching factor.